Máximo común divisor (MCD) + Mínimo común múltiplo (MCM)

Informe de Laboratorio N° 3: Implementación y Análisis en LC-3

Alumnos:

Alvaro Martin Zabala

Pedro Sebastian Behr

Grupo 7

Materia:

Arquitectura de Computadoras

Tema:

Algoritmos de MCD y MCM en Lenguaje Ensamblador (LC-3)

2026

(Aclaramos que el programa en assembly ‘mcd_mcm_v3.asm’ es un programa autónomo, es decir, es independiente del ‘main_menu.asm’ subido por el grupo 27; sin embargo, este programa por sí solo funciona en cualquier simulador de la LC-3, incluyendo la extensión de Visual Studio Code).

1. Bitácora de Investigación

Se sigue el formato de registro diario de la guía del laboratorio. Cada sesión indica la fecha, el objetivo, las actividades, los hallazgos, las dificultades y el plan siguiente.

Sesión 1

Fecha y hora: 14 de octubre de 2026

Objetivo del día: Estudiar el algoritmo del MCD y del MCM en lenguaje de alto nivel (C), escribir la rutina en assembly de la LC-3 y armar un simulador en C que ejecute el código y mida su desempeño.

Actividades realizadas:

- Se definió cómo calcular MCD y MCM con lo que ofrece la LC-3: solo suma (ADD), AND bit a bit y negación (NOT). No tiene instrucciones de resta, multiplicación ni división.
- Se programó un modelo en C que cuenta las iteraciones de cada bucle antes de pasar a bajo nivel.
- Se tradujo el algoritmo a assembly (mcd_mcm_v1.asm): valor absoluto de las entradas negativas, manejo del cero y detección de overflow en 16 bits.
- Se programó un simulador funcional en C que carga el código desde x3000, permite probar pares de números por consola e informa la memoria usada y las instrucciones ejecutadas por opcode. Implementa solo las instrucciones que usa el programa (no incluye RTI ni otros TRAP que no sean HALT).

Observaciones y hallazgos:

- **Conviene dividir antes de multiplicar.** La fórmula clásica MCM(A, B) = |A · B| / MCD(A, B) multiplica primero, y casi cualquier par de números medianos supera los 32767 que admite un entero de 16 bits con signo. Ejemplo: con A = 200 y B = 300 el producto es 60000 (overflow), aunque el MCM real es solo 600. Si se calcula primero Q = |A| / MCD y después Q · |B|, el resultado solo desborda cuando el MCM real no entra en 16 bits.
- **La resta sale cara.** Sin instrucción SUB, cada resta necesita 3 instrucciones (NOT, ADD #1 y ADD). Esto pesa cuando A y B están muy alejados entre sí, y marca qué optimizar en la nueva ISA de 5 bits de opcode.

Dificultades u obstáculos encontrados y cómo se resolvieron:

- **Error del ensamblador con TRAP x25.** El primer borrador, en el simulador externo, dio el error “Use of trap instruction with explicit vector operand”. El ensamblador no acepta el vector explícito y pide el alias HALT. **Solución:** se reemplazaron las dos apariciones de TRAP x25 por HALT, que ensambla al mismo código máquina (0xF025).
- **Poca comodidad para probar varios casos.** Los valores estaban fijos en el código o se pasaban como argumentos. **Solución:** se agregó un bucle interactivo con scanf y una validación del rango [-32767, 32767].

Plan para la siguiente sesión:

- Evaluar el efecto de agregar SUB (resta directa) y SHR (desplazamiento a la derecha) en la nueva ISA, y comparar Euclides con el algoritmo binario de Stein.

Sesión 2 (cambio de v1 a v2)

Fecha: 20 de octubre de 2026

Objetivo del día: Revisar el código, probarlo exhaustivamente con casos de borde y validar la coincidencia entre el modelo en C y el código en Assembly.

Actividades realizadas:

- Se revisó la rutina en assembly para asegurar la correcta preservación de registros y el manejo de casos límite.
- Se agregó al programa en C un modo de pruebas automatizadas (mainv3 --test) para verificar el comportamiento del algoritmo frente a una referencia independiente de 64 bits.
- Se probaron 49.370 pares: valores borde, una grilla de -60 a 60, 15.000 pares aleatorios y pares con MCD grande.

Observaciones y hallazgos:

- **Error corregido con cero.** Si una entrada era 0, el programa autónomo y el modelo en C devolvían MCD = 0. Lo correcto es MCD(0, B) = |B| (y MCD(0, 0) = 0); el MCM sí vale 0. La primera corrida de pruebas dio 285 fallas, todas por este motivo; con la corrección quedaron 0 fallas en los 49.370 pares.
- **Ejemplo mal calculado en el informe anterior.** MCM(300, 400) es 1200 y no desborda. Se lo reemplazó por el par 1000 y 999, cuyo MCM real (999000) sí desborda.
- **Comportamiento ante overflow en el MCM:** Si el resultado supera 32767 en la versión base sin corte temprano, devuelve el valor módulo 65536 (por ejemplo, 15960 para 1000 y 999) y registra el desborde mediante FLAG_OVF.
- **FLAG_OVF es un contador.** Cualquier valor distinto de 0 indica overflow. Para 1000 y 999 vale 492, no 1.

Dificultades u obstáculos encontrados y cómo se resolvieron:

- **Falta probar en las herramientas oficiales.** Las pruebas usaron un ensamblador y un simulador propios, no los de la cátedra. Queda pendiente.

Plan para la siguiente sesión:

- Ensamblar y probar el programa interactivo en el simulador oficial de la cátedra.
- Avisar el overflow del MCM devolviendo 0x8000 (-32768), un valor que nunca es un MCM válido, y cortar la multiplicación apenas se detecta.
- Mejorar el rendimiento usando como contador el menor de los dos factores.
- Comparar con el algoritmo binario de Stein (SUB y SHR).

Sesión 3

Fecha y hora: 26 de octubre de 2026.

Objetivo del día: Desarrollar y depurar un programa de prueba interactivo por consola (mcd_mcm_v2.asm), verificar su ensamblado en el simulador web oficial de la cátedra y ejecutar una batería de 10 casos de prueba extremos (incluyendo números negativos, ceros, desbordamientos y límites de 16 bits).

Actividades realizadas:

- Se programó la versión interactiva mcd_mcm_v2.asm, incorporando las subrutinas LEER_ENTERO (lectura de enteros decimales con signo desde el teclado) e IMPRIMIR_ENTERO (conversión y salida decimal en pantalla).
- Se ejecutaron 10 pruebas de estrés en el simulador web oficial, inspeccionando las posiciones de memoria RESULTADO_MCD (x30ED) y RESULTADO_MCM (x30EE).

Observaciones y hallazgos:

- **Verificación exitosa de las 10 pruebas en el simulador web:**
  - **Caso 1 (48 y 180):** MCD = 12 (x000C), MCM = 720 (x02D0).
  - **Caso 2 (-48 y 180):** MCD = 12 (x000C), MCM = 720 (x02D0). Se confirma la correcta normalización de las entradas negativas.
  - **Caso 3 (200 y 300):** MCD = 100 (x0064), MCM = 600 (x0258). Se evita el desbordamiento prematuro al dividir antes de multiplicar.

- **Caso 4 (0 y -25):** MCD = 25 (x0019), MCM = 0 (x0000). Se valida la propiedad MCD(0, B) = |B|.
- **Caso 5 (1234 y 1234):** MCD = 1234 (x04D2), MCM = 1234 (x04D2). Salida directa en la primera iteración del algoritmo de Euclides.
- **Caso 6 (1000 y 999):** MCD = 1 (x0001), MCM = 15960 (x3E58, equivalente a 999 000 mod 65 536).
- **Caso 7 (200 y 199):** MCD = 1 (x0001), MCM = -25736 (x9B78, equivalente al resultado interpretado en complemento a dos de 16 bits).
- **Caso 8 (32767 y 1):** MCD = 1 (x0001), MCM = 32767 (x7FFF). Prueba de límite superior positivo.
- **Caso 9 (-32767 y -32766):** MCD = 1 (x0001), MCM = -32766 (x8002, dado que 32 767 × 32 766 mod 65 536 = 32 770).
- **Caso 10 (-32768 y 10):** MCD = 2 (x0002), MCM = -32768 (x8000). Aunque -32768 se encuentra fuera del dominio especificado, la resta inicial de Euclides (x8000 - 10 = x7FF6 = +32758) absorbe el desbordamiento en complemento a dos e ingresa el valor en el rango positivo sin bloquear la ejecución.

Dificultades u obstáculos encontrados y cómo se resolvieron:

- **Error con números negativos en LEER_ENTERO:** En la versión inicial de mcd_mcm_v2.asm, la subrutina ignoraba el signo menos (-). Se identificó que la instrucción ADD R5, R0, #0, ubicada inmediatamente después de GETC, sobrescribía el contador de caracteres (R5) con el código ASCII del guion (45), impidiendo evaluar la condición R5 == 0. **Solución:** Se eliminó esa instrucción y se añadió ADD R5, R5, #1 en SIGNO_NEGATIVO para registrar adecuadamente la lectura del primer carácter.
- **Uso de vectores TRAP explícitos e inicialización de R6:** El borrador empleaba instrucciones directas como TRAP x20 o LD R6, STACK_BASE. **Solución:** Se sustituyen por las pseudoinstrucciones estándar (GETC, OUT, PUTS, HALT) para garantizar total compatibilidad con el ensamblador oficial, y se omitió la carga manual de R6 dado que el simulador lo gestiona automáticamente.

Plan para la siguiente sesión:

- Definir con el docente a cargo (Ing. Eduardo Díaz) si en el cálculo del MCM se mantendrá el valor retornado módulo 65536 o si se devolverá 0x8000 (-32768) como aviso ante un desbordamiento.

Sesión 4 (Optimización, cambio de v2 a v3)

Fecha y hora: 27 de octubre de 2026.

Objetivo del día: Analizar el comportamiento aritmético del límite inferior en complemento a dos (-32768 / x8000) y optimizar el rendimiento en bajo nivel del código en assembly LC-3 (ahora mcd_mcm_v3.asm) sin alterar los algoritmos matemáticos base.

Actividades realizadas:

- Se investigó por qué el par (-32768, 2) no produce bloqueo en el código original y devuelve MCD = 2 (x0002) y MCM = -32768 (x8000). Se comprobó que en la primera resta de Euclides (x8000 - 2 = x7FFE = +32766) el underflow en complemento a dos corrige el signo del operando hacia el rango positivo válido, y que el MCM matemático real (+32768) coincide bit a bit en binario de 16 bits con x8000 (-32768 con signo).
- A partir de las métricas del simulador funcional en C (mainv3.c), donde casos extremos como (32767, 1) ejecutaban 458.754 instrucciones y (-32767, -32766) alcanzaban 524.296 instrucciones, se implementaron cuatro optimizaciones estructurales en assembly:
  - **Ordenamiento inicial de valores absolutos:** asignación de R4 = min(|A|, |B|) como dividendo para hallar el cociente Q y R5 = max(|A|, |B|) como multiplicando, reduciendo drásticamente las iteraciones de división y multiplicación.

  - **Eliminación del recálculo redundante de complemento a dos:** (NOT + ADD #1) dentro del bucle de Euclides: se conserva el sustraendo negado en R2 durante toda la racha de restas consecutivas, bajando el costo del lazo interno de 7 a 3 instrucciones por iteración.
  - **Detección y corte temprano de desbordamiento (Early Exit):** en la multiplicación con valor centinela x8000 (-32768): apenas el acumulador supera 32767 (bit 15 = 1, condición N), se aborta el bucle de inmediato y se almacena x8000 en RESULTADO_MCM.
  - **Optimización del bucle de división:** eliminación de la instrucción redundante ADD R4, R4, #0 reordenando el incremento del cociente antes de la resta.

Observaciones y hallazgos:

- **Caso 1 (48 y 180):** Se mantiene en 15 iteraciones totales (7 + 4 + 4), pero las instrucciones del núcleo de cálculo bajan de 103 a ~68 (-34%).
- **Caso 3 con Overflow (1000 y 999):** Las iteraciones caen de 3.000 (1000 + 1000 + 1000) a 2.032 (1000 + 999 + 33), ya que MUL_LOOP detecta el desborde en la suma N° 33 y corta con el centinela -32768. Las instrucciones bajan de 16.504 a ~6.140 (-63%).
- **Caso 4 Peor Caso sin Overflow (32767 y 1):** Al dividir el menor (1 / 1 = 1 iteración) en vez del mayor (32767 / 1 = 32767 iteraciones), las iteraciones totales caen de 98.301 a 32.769 (32767 + 1 + 1), y las instrucciones ejecutadas se reducen de 458.754 a ~98.330 (-78,5%, ahorrando más de 360.000 instrucciones).
- **Peor Caso General (-32767 y -32766):** Gracias al bucle corto de Euclides y al corte temprano de overflow en la segunda suma, las iteraciones bajan de 98.301 a 65.535 y las instrucciones caen de 524.296 a ~196.630 (-62,5%, ahorrando más de 327.000 instrucciones).

Dificultades u obstáculos encontrados y cómo se resolvieron:

- **Error del ensamblador web "Unknown opcode 'MCD_LISTO'":** Al unificar las salidas del bucle optimizado de Euclides, habían quedado dos etiquetas consecutivas en líneas distintas apuntando a la misma instrucción. El ensamblador oficial interpretó la segunda etiqueta como una instrucción inválida. **Solución:** Se eliminó la etiqueta intermedia redundante y se redirigieron todos los saltos correspondientes directamente a MCD_LISTO, logrando un ensamblado correcto.

Plan para la siguiente sesión:

- Documentar formalmente el formato Q15.0, los rangos de entrada y salida, el manejo de overflow, las iteraciones y las métricas de desempeño finales para la entrega, verificar nuevamente el código en assembly y reorganizar el código en C.

Sesión 5

Fecha y hora: 30 de octubre de 2026

Objetivo del día: Separar la lógica de referencia en C de la simulación del programa autónomo LC-3 y corregir la gestión de la pila en Assembly.

Actividades realizadas:

- Se reorganizó mainv3.c en dos caminos independientes: cálculo exclusivo en C y ejecución en simulador LC-3.
- En mcd_mcm_v3.asm se inicializó R6 con STACK_BASE en xFDFF y se optimizó la pila eliminando instrucciones redundantes.

Observaciones y hallazgos:

- La comparación automática alcanzó 63.750 pares de entrada válidos sin fallas (coincidencia completa entre C y LC-3).

Dificultades u obstáculos encontrados y cómo se resolvieron:

- **Sobreescritura del valor B en el modelo en C:** El modelo modificaba B durante Euclides. **Solución:** Se preservó una copia de la entrada original para validar correctamente.

Plan para la siguiente sesión:

Continuar con la documentación del informe y realizar consultas acerca de la implementación de los algoritmos “Multiplicación por shift” y “división larga” para nuestro algoritmo.

2. Informe de Avance — Semana 1 (Análisis del Algoritmo)

Se detallan los puntos pedidos para la presentación de avance, sobre el programa en lenguaje de alto nivel.

2.1. Objetivo e Historia del Algoritmo

- **Objetivo:** dados dos enteros A y B, obtener su Máximo Común Divisor (MCD), que es el mayor entero positivo que divide a ambos sin dejar resto, y su Mínimo Común Múltiplo (MCM), que es el menor múltiplo positivo común a ambos.
- **Quién lo propuso:** el MCD se calcula con el Algoritmo de Euclides, publicado por Euclides de Alejandría hacia el año 300 a.C. en el Libro VII (proposiciones 1 y 2) de su obra Elementos. Es uno de los algoritmos más antiguos que se siguen usando. Euclides lo planteó de forma geométrica: para hallar la “medida común máxima” de dos segmentos, restaba el menor del mayor una y otra vez hasta que ambos fueran iguales.

2.2. Funcionamiento Paso a Paso

El algoritmo tiene cuatro etapas, una después de la otra:

1. **Normalización.** Si una entrada es 0, el MCM es 0 y el MCD es el valor absoluto de la otra entrada (MCD(0, 0) = 0). Si A o B son negativos, se pasan a positivos con complemento a 2: |X| = ~X + 1.
2. **Cálculo del MCD (Euclides por restas sucesivas).** Se comparan |A| y |B|:
   - Si |A| > |B|, se hace |A| = |A| − |B|.
   - Si |B| > |A|, se hace |B| = |B| − |A|.
   - Cuando |A| = |B|, el bucle termina y ese valor es el MCD.
3. **División exacta por restas sucesivas (Q = |A| / MCD).** Se empieza con un contador Q = 0. En cada vuelta se resta el MCD al valor original de |A| y se suma 1 a Q, hasta llegar a 0. Como el MCD divide a |A|, el resto siempre es 0.
4. **Multiplicación por sumas sucesivas (MCM = Q · |B|).** Se suma |B| consigo mismo Q veces. En cada suma se mira el bit de signo del acumulador: si al sumar números positivos el resultado pasa a negativo (bit 15 en 1), el MCM superó 32767 y se activa la bandera FLAG_OVF.

2.3. Formato de Punto Fijo, Rango y Precisión

El MCD y el MCM trabajan con divisores y múltiplos enteros. Por eso se usa un entero con signo de 16 bits, sin parte fraccionaria:

Parámetro	Especificación adoptada	Justificación
Formato de punto fijo	Q15.0 (complemento a 2, 16 bits)	1 bit de signo y 15 bits de parte entera; 0 bits fraccionarios.
Rango de entrada (A, B)	[-32767, 32767]	Se excluye -32768 (0x8000) porque su valor absoluto (32768) no cabe en Q15.0.
Rango de salida (MCD)	[0, 32767]	Nunca desborda: el MCD es menor o igual que el mayor de |A| y |B|, que a su vez es menor o igual que 32767.

Parámetro	Especificación adoptada	Justificación
Rango de salida (MCM)	[0, 32767], exacto	En la versión optimizada (mcd_mcm_v3.asm), si el MCM real supera 32767 corta de inmediato y devuelve el centinela -32768 (0x8000) como aviso de overflow; en el programa autónomo base queda módulo 65536 con FLAG_OVF != 0.
Precisión (mínima y máxima)	Exacta: resolución 2^0 = 1, error 0	No hay truncamiento ni redondeo, porque |A| / MCD siempre es una división exacta (salvo en el MCM con overflow).

2.4. Análisis de Iteraciones y Ejemplos de Prueba

La cantidad de iteraciones depende de la relación entre A y B:

- **Bucle MCD:** mínimo 1 iteración (cuando |A| = |B|) y máximo max(|A|, |B|) (peor caso: un número vale 32767 y el otro 1). El modelo en C cuenta la comparación final como una iteración.
- **Bucle de división:** exactamente Q = |A| / MCD iteraciones.
- **Bucle de multiplicación:** exactamente Q iteraciones.

Los cuatro ejemplos siguientes se midieron en el simulador en C sobre el programa autónomo base (previo a las cuatro optimizaciones estructurales de mcd_mcm_v3.asm documentadas en la Sesión 4). Se comparan las iteraciones del modelo de alto nivel con las instrucciones LC-3 ejecutadas (Instr.) en el programa autónomo.

Caso de prueba	Entradas (A, B)	Resultados (MCD / MCM)	Iteraciones (MCD + DIV + MUL)	Instr. (autónomo)	Observación
1. Estándar	48 y 180	12 / 720	7 + 4 + 4 = 15	103	Caso típico; ambos resultados entran de sobra en 16 bits.
2. Overflow evitado	200 y 300	100 / 600	3 + 2 + 2 = 7	53	|200 · 300| = 60000 desbordaría, pero al dividir primero (200 / 100 = 2) no hay overflow.
3. Overflow real	1000 y 999	1 / 15960 (exacto: 999000)	1000 + 1000 + 1000 = 3000	16.504	El MCM real no entra en 16 bits: queda módulo 65536 y FLAG_OVF ≠ 0 (vale 492).
4. Peor caso	32767 y 1	1 / 32767	32767 × 3 = 98301	458.754	Caso límite del benchmark: muestra el costo de restar de a 1 en Euclides.

El mayor costo medido en las pruebas previas a la optimización fue con el par (-32767, -32766). Es un máximo medido empíricamente, no una cota teórica demostrada.

2.5. Verificación

- Se estructuró el programa en C (mainv3.c) en dos implementaciones independientes para validación cruzada: calcular_solo_c (lógica pura en C en formato Q15.0 mediante sumas y restas sucesivas) y ejecutar_programa_lc3 (simulación funcional ciclo a ciclo del código máquina autónomo de 53 palabras cargado desde x3000 hasta HALT).
- Mediante el modo de prueba automatizado (mainv3 --test), se verificaron 63.750 pares de entrada válidos en el rango [-32767, 32767] obteniendo 0 fallas y coincidencia exacta (100%) entre el modelo en C y el simulador LC-3 tanto en el MCD, el MCM como en la detección de overflow.
- Se validó explícitamente el rechazo de la entrada fuera de dominio -32768 (INT16_MIN / x8000) en C (entrada_invalida = true) y el uso de x8000 como valor centinela de corte temprano por overflow en la versión interactiva optimizada en Assembly.

3. Código Desarrollado

3.1. Programa interactivo optimizado en Assembly (mcd_mcm_v3.asm) y Subrutinas Principales

El archivo mcd_mcm_v3.asm implementa el algoritmo de sumas y restas sucesivas en la ISA estándar de la LC-3, incorporando optimizaciones estructurales a nivel de registros y las subrutinas de entrada/salida por consola:

- **Normalización y ordenamiento inicial (A_POSITIVO, B_POSITIVO, A_MAYOR_IGUAL_B):** Convierte las entradas A y B a valor absoluto mediante complemento a dos (NOT + ADD #1), resuelve los casos borde donde alguna entrada es cero (A_ES_CERO, B_ES_CERO) y ordena los operandos asignando R4 = min(|A|, |B|) y R5 = max(|A|, |B|). Además, precalcula -|B| en R2.
- **Núcleo del MCD por restas sucesivas optimizado (INICIAR_MCD, RESTAR_A_R0, RESTAR_A_R1):** Ejecuta el algoritmo de Euclides conservando el sustraendo negado en R2 durante toda una racha de restas del mismo signo. Esto reduce el bucle interno a solo 3 instrucciones por iteración (ADD, ADD, BRp) sin recalcular NOT + ADD #1 en cada paso.
- **División exacta por restas sucesivas (DIV_LOOP):** Calcula el cociente exacto Q = min(|A|, |B|) / MCD restando el MCD (negado en R3) al operando menor (R4) e incrementando el contador Q (R2). Al dividir el menor de los números, minimiza la cantidad de iteraciones.
- **Multiplicación por sumas sucesivas con corte temprano (MUL_LOOP, MCM_OVERFLOW):** Calcula MCM = Q * max(|A|, |B|) sumando R5 consigo mismo Q veces en R0. En cada suma evalúa el flag de condición negativo (BRn MCM_OVERFLOW): si el acumulador supera 32767 (bit 15 en 1), aborta inmediatamente el bucle y carga el centinela x8000 (-32768) en RESULTADO_MCM.
- **Subrutina LEER_ENTERO:** Lee caracteres ASCII desde el teclado con GETC y hace eco con OUT hasta detectar un salto de línea (LF o CR). Conserva R7 en la pila (R6), valida el signo negativo (-) únicamente en el primer carácter (cuando R5 == 0) y acumula el valor decimal multiplicando el acumulador previo por 10 mediante sumas de desplazamientos (R1 * 10 = R1 * 2 + R1 * 8) antes de sumar el nuevo dígito. Si el número era negativo (R4 < 0), aplica complemento a dos al finalizar.
- **Subrutina IMPRIMIR_ENTERO:** Recibe un entero con signo de 16 bits en R0 y lo imprime en decimal por consola. Apila R7, imprime el signo '-' si es negativo (tratando de forma especial el valor x8000 mediante la cadena "32768" en TEXTO_MIN_INT) y extrae cada dígito decimal por restas sucesivas contra una tabla de potencias de 10 (POTENCIAS: 10000, 1000, 100, 10, 1), suprimiendo los ceros a la izquierda.

3.2. Modelo de Referencia y Simulador Funcional en C (mainv3.c)

El programa en C (mainv3.c) cumple dos funciones esenciales para el laboratorio:

- **calcular_solo_c(int16_t a_entrada, int16_t b_entrada):** Implementa la lógica matemática pura en formato Q15.0 utilizando exclusivamente restas sucesivas para el MCD y la división, y sumas sucesivas para el MCM. Preserva copias de los valores absolutos originales (original_a y original_b), contabiliza las iteraciones de cada etapa (iter_mcd, iter_div, iter_mul) y detecta desbordamientos cuando el acumulador supera INT16_MAX (32767), activando overflow_mcm = true y conservando el resultado módulo 65536 en coincidencia con el programa autónomo simulado.
- **ejecutar_programa_lc3(...):** Implementa una máquina virtual de la arquitectura LC-3 con 65.536 palabras de memoria, 8 registros (R0–R7), registro PC y flags de condición (N, Z, P). Carga el arreglo PROGRAMA_AUTONOMO en x3000, inyecta las entradas en x3030 y x3031, ejecuta instrucción por instrucción contabilizando el uso de cada opcode y extrae los resultados de x3032 (MCD), x3033 (MCM) y x3034 (flag de overflow).
- **ejecutar_pruebas():** Recorre automáticamente una grilla de 63.750 combinaciones en el rango [-32767, 32767] verificando mediante la función iguales() que los resultados de C y de la LC-3 coincidan sin discrepancias.

4. Limitaciones y Casos No Cubiertos

- **Entrada -32768:** está fuera del dominio Q15.0 porque su valor absoluto (32768) no cabe en 16 bits con signo. En el programa en C (mainv3.c) se rechaza explícitamente (entrada_invalida = true), mientras que en la rutina en Assembly no se valida en la entrada.
- **Overflow del MCM:** en la versión interactiva optimizada (mcd_mcm_v3.asm), si el MCM supera 32767 el bucle corta de inmediato y devuelve el valor centinela -32768 (x8000). En la versión base del simulador sin corte temprano, devuelve el valor módulo 65536 (por ejemplo, MCM(1000, 999) devuelve 15960 en lugar de 999000) activando FLAG_OVF.
- **Rendimiento:** Euclides por restas necesita del orden de max(|A|, |B|) pasos, porque la LC-3 no tiene división. Se cuentan instrucciones, no ciclos ni tiempo.
- **Uso de la pila:** las subrutinas de entrada/salida (LEER_ENTERO e IMPRIMIR_ENTERO) requieren que el puntero de pila R6 esté inicializado para resguardar la dirección de retorno (R7).
- **Herramientas oficiales:** El algoritmo y las subrutinas de E/S fueron ensamblados y verificados exitosamente en el simulador web oficial de la cátedra.
- **Pruebas:** La validación automática en mainv3.c (--test) cubre una grilla uniforme de 63.750 pares de entrada válidos con 0 fallas (además de las baterías de casos borde probadas en sesiones previas).
- **Simulador en C:** implementa solo las instrucciones que usa el programa. No tiene RTI, ningún TRAP salvo HALT, ni entrada ni salida.
- **Fuera de alcance:** MCD y MCM de más de dos números, MCD extendido (coeficientes de Bézout), MCM de más de 16 bits, valores con parte fraccionaria y simplificación de fracciones.
- **Uso de IA:** Parte del análisis inicial, del simulador en C y de la batería de pruebas automatizadas se agilizó con asistencia de modelos de lenguaje (IA generativa), verificando, depurando y reproduciendo manualmente cada resultado tanto en el modelo en C (mainv3 --test) como en el simulador oficial de la LC-3.